

Uniwersytet Gdański
Wydział Matematyki, Fizyki i Informatyki

Dawid Kacper Odolczyk
nr albumu: 298988
kierunek studiów: informatyka

Analiza porównawcza i integracyjna
mechanizmów Project Loom
z istniejącymi modelami
współbieżności w środowisku JVM
w kontekście wybranych scenariuszy
zastosowań

Praca magisterska
wykonana pod kierunkiem
dra Tomasza Borzyszkowskiego

Gdańsk 2026

Spis treści

Streszczenie	5
1 Wstęp	7
1.1 Motywacja	7
1.2 Cel i zakres pracy	7
1.3 Pytania badawcze	7
1.4 Struktura pracy	7
2 Wprowadzenie teoretyczne	9
2.1 Podstawowe pojęcia	9
2.1.1 Współbieżność a równoległość	9
2.1.2 Pojęcie i zasada działania wątków	10
2.2 Problemy występujące w systemach współbieżnych	11
2.2.1 <i>Race condition</i> i pamięć współdzielona	11
2.2.2 Problemy żywotności: <i>zakleszczenie</i> , <i>zagłodzenie</i> , <i>livelock</i>	13
2.2.3 Problemy związane z wydajnością	15
2.3 Modele współbieżności w środowisku JVM	15
2.3.1 Klasyczny model współbieżności (Thread API)	15
2.3.2 Model oparty o pulę wątków (<code>ExecutorService</code>)	16
2.3.3 Reaktywny model współbieżności (Reactive Streams, Project Reactor, RxJava)	17
3 Mechanizmy współbieżności oferowane przez Project Loom	19
3.1 Wprowadzenie do Project Loom	19
3.2 Wirtualne wątki (<i>Virtual Threads</i>)	20
3.3 Współbieżność strukturalna (<i>Structured Concurrency</i>)	22
3.4 Zmienne zakresowe (<i>Scoped Values</i>)	22
4 Stan badań i analiza literatury	23
4.1 Dotychczasowe badania nad Project Loom	23
4.2 Charakterystyka i wyniki dotychczasowych badań	23
4.2.1 Obsługa dużej liczby równoczesnych żądań HTTP	23

4.2.2	Równoległe przetwarzanie złożonych obliczeń	23
4.3	Zdefiniowanie luki badawczej i kierunku własnych badań	23
5	Metodyka badań	25
5.1	Cel i zakres badań	25
5.2	Szczegółowy opis scenariuszy testowych	25
5.2.1	Scenariusz A: Integracja z bazą danych ze sterownikiem blokującym (JDBC) i reaktywnym (R2DBC)	25
5.2.2	Scenariusz B: Adaptacyjna strategia Hedged Requests z prio- rytetyzacją jakości danych i dynamiczną propagacją parame- trów SLA	25
6	Wyniki badań i analiza	27
6.1	Metody pomiaru i kryteria oceny	27
6.2	Wyniki pomiarów dla poszczególnych scenariuszy	27
6.3	Analiza porównawcza i interpretacja wyników	27
7	Wnioski i rekomendacje	35
7.1	Odpowiedzi na pytania badawcze	35
7.2	Rekomendacje i wzorce zastosowań mechanizmów Project Loom . . .	35
7.3	Ograniczenia badań i propozycje dalszych badań	35
8	Podsumowanie	37
	Bibliografia	39
	Spis rysunków	41
	Spis tabel	42
	Spis kodów źródłowych	43

Streszczenie

Celem niniejszej pracy jest ocena wpływu mechanizmów wirtualnych wątków i współbieżności strukturalnej, będących częścią inicjatywy Project Loom, na wydajność oraz złożoność implementacyjną aplikacji współbieżnych w środowisku JVM. W pracy analizowane są trzy główne podejścia do współbieżności: wirtualne wątki (Project Loom), Reactive Streams oraz Kotlin Coroutines. Badanie opiera się na realistycznych scenariuszach produkcyjnych, obejmujących obsługę równoczesnych żądań HTTP, integrację z zewnętrznymi API, przetwarzanie danych w bazach danych oraz intensywne przetwarzanie strumieniowe danych telemetrycznych.

Praca koncentruje się zarówno na aspektach technicznych (przepustowość, opóźnienia, zużycie zasobów), jak i implementacyjnych (czytelność kodu, złożoność architektury, propagacja kontekstu i błędów). Wyniki testów pozwalają na ocenę, w jakim stopniu wirtualne wątki upraszczają projektowanie aplikacji współbieżnych, a także na sformułowanie praktycznych rekomendacji dla zespołów developerskich rozważających migrację do Loom lub hybrydowe podejścia współbieżności w JVM.

Abstract

“A Comparative and Integrative Analysis of Project Loom Mechanisms and Existing Concurrency Models in the JVM Environment for Selected Application Scenarios”

The engineering thesis describes the development of a mobile application made using the Flutter framework. The application is designed for scanning barcodes of food products to obtain detailed information about them. In addition to providing product information such as ingredients, allergens presence, and nutritional value, the application allows users to determine if a product is suitable for them based on their dietary restrictions and preferences. The application features a system for retrieving product information, which reduces the need to query external APIs. Furthermore, the application does not require an Internet connection to be functional.

The thesis begins by addressing the need for easier access to information about food products' ingredients, especially among individuals with food intolerances. Subsequently, the thesis provides a detailed description of the items present on food products' labels, such as the structure of EAN barcodes, types of allergens and nutritional values along with standards for their presentation, and a detailed description of the Nutri-Score system. Next, it describes the technologies used in developing the application and the data structure it relies on, including information about products available in the Open Food Facts database. The thesis further focuses on the application's architecture, programming practices used during its development, including widget-based architecture with detailed descriptions of their functionality, as well as the process of retrieving and storing product information. The thesis concludes with insights into the application's development process and suggestions for future improvements.

Rozdział 1

Wstęp

```
Runnable task = () -> System.out.println("Hello, World!");  
Thread.startVirtualThread(task);
```

Listing 1: Uruchamianie wirtualnego wątku
(źródło: opracowanie własne na podstawie X)

1.1 Motywacja

1.2 Cel i zakres pracy

1.3 Pytania badawcze

1.4 Struktura pracy

Rozdział 2

Wprowadzenie teoretyczne

Drugi rozdział pracy poświęcony jest teoretycznemu omówieniu podstawowych pojęć związanych z równoległością i współbieżnością, w tym w omówieniu działania wątków we współczesnych systemach operacyjnych, zdefiniowaniu typowych problemów występujących w systemach współbieżnych oraz przedstawieniu najważniejszych założeń różnych modeli współbieżności dostępnych w ramach środowiska JVM, w szczególności klasycznego modelu współbieżności oferowanego w ramach Thread API, modelu opartego o pulę wątków (dostępnego w ramach interfejsu `ExecutorService`) oraz reaktywnego modelu współbieżności oferowanego przez Reactive Streams i RxJava.

2.1 Podstawowe pojęcia

2.1.1 Współbieżność a równoległość

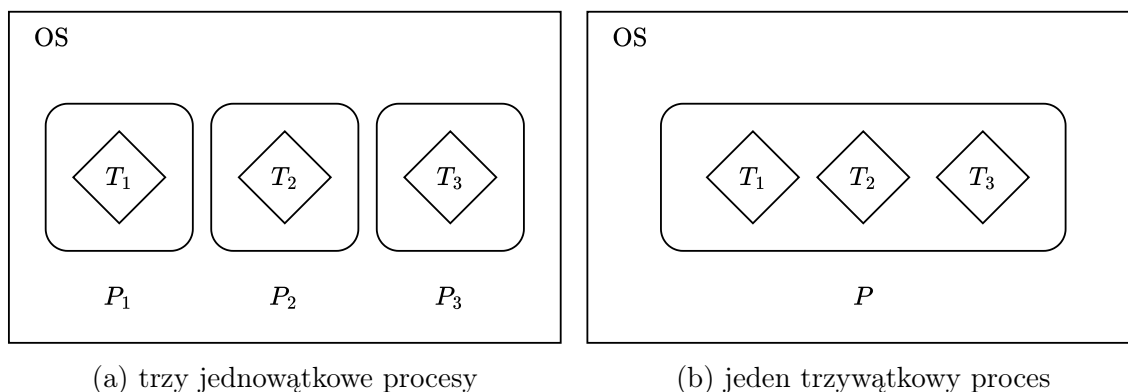
Jak podają Tanenbaum i Bos [1], *współbieżność* (ang. *concurrency*) oznacza zdolność systemu do obsługi wielu zadań w taki sposób, że ich wykonywanie nakłada się w czasie, nawet jeśli w danym momencie wykonywane jest tylko jedno z nich. *Równoległość* (ang. *parallelism*) oznacza obsługę wielu zadań w tym samym czasie przy wykorzystaniu wielu jednostek obliczeniowych.

Innymi słowy, współbieżność odnosi się do strukturalnej organizacji systemu oraz sposobu modelowania i zarządzania wieloma zadaniami, natomiast równoległość oznacza ich faktyczne, jednoczesne wykonywanie na wielu procesorach lub rdzeniach. We współczesnych systemach operacyjnych zadania te są wykonywane w ramach *procesów* i *wątków*.

2.1.2 Pojęcie i zasada działania wątków

We współczesnych systemach operacyjnych najważniejszą abstrakcją umożliwiającą współbieżne wykonywanie operacji są *procesy* (ang. *processes*). Jak wskazują Tanenbaum i Bos [1], procesy umożliwiają przekształcenie jednej jednostki obliczeniowej (CPU) na wiele wirtualnych jednostek, w ramach których mogą współbieżnie wykonywać zadania poprzez umiejętność CPU do szybkiego przełączania się między procesami.

W celu umożliwienia bardziej efektywnej współbieżności wewnątrz pojedynczego procesu, współczesne systemy operacyjne oferują *wątki* (ang. *threads*), definiowane jako lżejsze jednostki wykonania. Wątki działają w obrębie jednego procesu i współdzielą z nim przestrzeń adresową oraz zasoby, zachowując jednocześnie własny kontekst wykonania [1]. Różnice między procesem a wątkiem prezentuje rysunek 2.1.



Rysunek 2.1: Rozróżnienie procesów i wątków w systemie operacyjnym

(źródło: opracowanie własne na podstawie [1])

Mimo że oba systemy przedstawione powyżej operują na trzech wątkach T_1 , T_2 i T_3 , wątki na rysunku 2.1a operują na odrębnych przestrzeniach adresowych, podczas gdy wątki przedstawione na rysunku 2.1b działają w obrębie jednej (wspólnej) przestrzeni adresowej, co wynika z faktu operowania wewnątrz jednego (wspólnego) procesu P .

Wprowadzenie wątków jako lekkich jednostek wykonania znacząco zwiększa efektywność wykorzystania zasobów systemowych, jednak jednocześnie wprowadza nowe wyzwania związane z koniecznością synchronizacji dostępu do współdzielonych danych. Współdzielenie przestrzeni adresowej przez wątki powoduje ryzyko występowania różnego rodzaju zjawisk, opisanych szczegółowo w podrozdziale 2.2 Problemy występujące w systemach współbieżnych. Z tego względu poprawne projektowanie systemów wielowątkowych wymaga nie tylko zrozumienia modelu wykonania wątków, ale również potencjalnych problemów wynikających z ich współbieżnej natury.

2.2 Problemy występujące w systemach współbieżnych

Mimo wielu zalet wykorzystania wątków do współbieżnego wykonywania zadań, podejście to wiąże się z istnieniem wielu problemów obecnych w systemach współbieżnych. Ben-Ari [2] wskazuje zasadę *wzajemnego wykluczenia* (ang. *mutual exclusion*) jako jeden z najważniejszych elementów programowania współbieżnego z racji bycia abstrakcją dla wielu problemów związanych z zarządzaniem wątkami. Zgodnie z definicją Herlihy’ego i Shavita [3], wątki *wykluczają się wzajemnie*, jeśli co najmniej dwa z nich próbują równocześnie wykonać wspólny blok kodu, nazywany *sekcją krytyczną* (ang. *critical section*), ale w danym momencie tylko jeden z nich ma do niej dostęp. Tę cechę systemu określa się również mianem *synchronizacji* wątków [4]. Oznacza to, że dany wątek ma dostęp do sekcji krytycznej tylko wtedy, gdy inny wątek zakończy w niej swoje operacje.

Zapewnienie tej cechy jest istotne w kontekście poprawności i jednoznaczności wyniku wielowątkowego zadania, a jej brak powoduje występowanie wielu kategorii problemów. Wśród nich można wyróżnić: problem *race condition* i pamięci współdzielonej, problemy żywotności, takie jak *zakleszczenie* (ang. *deadlock*), *zagłodzenie* (ang. *starvation*) i *livelock*, a także problemy związane z wydajnością. Poniżej opisano charakterystykę każdego z nich.

2.2.1 *Race condition* i pamięć współdzielona

Goetz [5] definiuje problem *race condition* (w polskiej literaturze często określany mianem *warunku wyścigu* lub *stanu wyścigu* [6, 7]) jako zachowanie systemu współbieżnego, w którym poprawność obliczeń zależy od względnego czasu lub przeplatania wielu wątków przez środowisko wykonawcze. Innymi słowy, problem ten występuje, gdy uzyskanie oczekiwanego rezultatu jest zależne od zbiegu okoliczności.

Najczęstszym rodzajem tego problemu jest sytuacja, w której potencjalnie nieaktualna wartość jest wykorzystywana do dalszych obliczeń, co może prowadzić do niezamierzonych rezultatów. Poniższy przykład szczegółowo prezentuje to zjawisko.

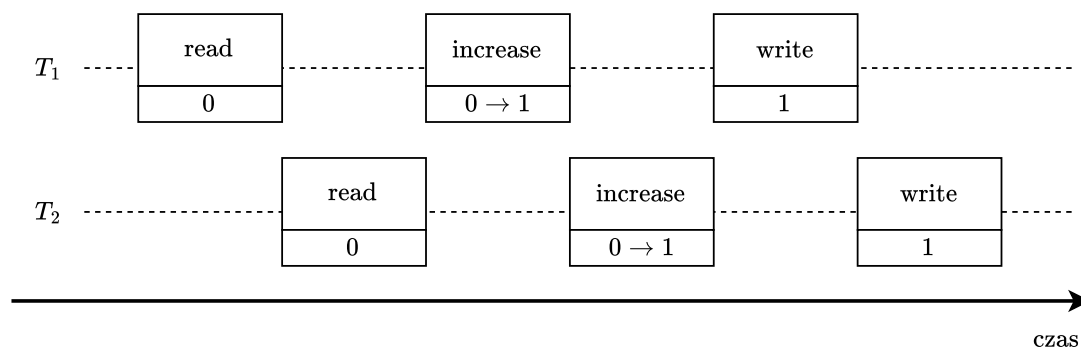
Niech procedura `increaseByOne` składa się z trzech wykonywanych sekwencyjnie operacji:

1. odczytania wartości zmiennej globalnej (ozn. `read`),
2. zwiększenia odczytanej wartości o 1 (ozn. `increase`),
3. zapisania nowej wartości do zmiennej globalnej (ozn. `write`).

Procedurę tę uruchomiono współbieżnie z wykorzystaniem wątków T_1 i T_2 dla tej samej wartości początkowej, tj. pewnej zmiennej globalnej G o wartości początkowej

równej 0. Przewidywanym wynikiem dwukrotnego wywołania procedury na zmiennej G jest zwiększenie wartości tej zmiennej do 2.

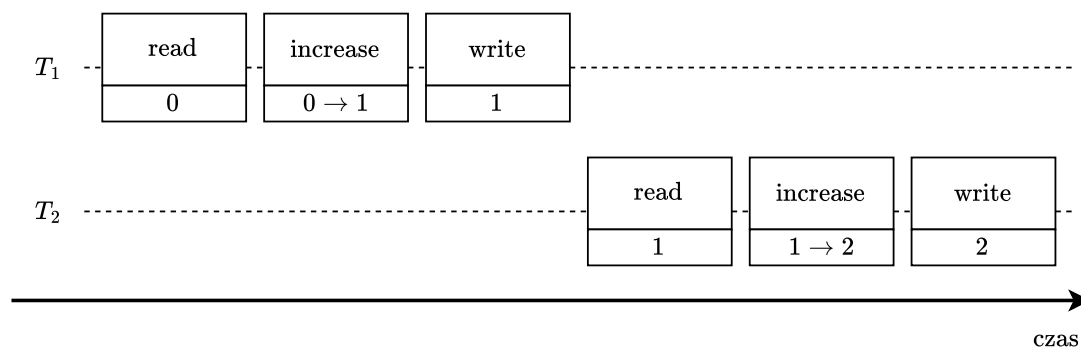
Rysunki 2.2 i 2.3 przedstawiają różne modele wywołań procedury `increaseByOne` na wątkach T_1 i T_2 . Prezentują one wywołania poszczególnych operacji tej procedury dla poszczególnych wątków w zależności od czasu. Każda operacja zawiera dwie informacje: nazwę oraz aktualną wartość zmiennej globalnej G . Wyjątek stanowi operacja `increase`, dla wartości której przyjęto oznaczenie „ $A \rightarrow B$ ”, gdzie A oznacza wartość zmiennej G przed dokonaniem na niej operacji zwiększenia wartości o 1 (tj. jej aktualna wartość), a B oznacza wartość zmiennej G po wykonaniu na niej operacji.



Rysunek 2.2: Ilustracja zjawiska *race condition* na podstawie wywołania procedury `increaseByOne`

(źródło: opracowanie własne na podstawie [5])

Rysunek 2.2 prezentuje jeden z przypadków wystąpienia zjawiska *race condition*, w którym poszczególne operacje procedury `increaseByOne` przeplatają się w czasie. W rezultacie wątek T_2 odczytał wartość zmiennej globalnej G w momencie, gdy nie została ona jeszcze zaktualizowana przez wątek T_1 , co w konsekwencji spowodowało, że wartość zmiennej G wynosi 1 zamiast przewidywanej wartości 2. Rozwiązaniem tego problemu jest wcześniej wspomniana synchronizacja wątków T_1 i T_2 .



Rysunek 2.3: Wywołanie procedury `increaseByOne` z wykorzystaniem synchronizacji wątków

(źródło: opracowanie własne na podstawie [5])

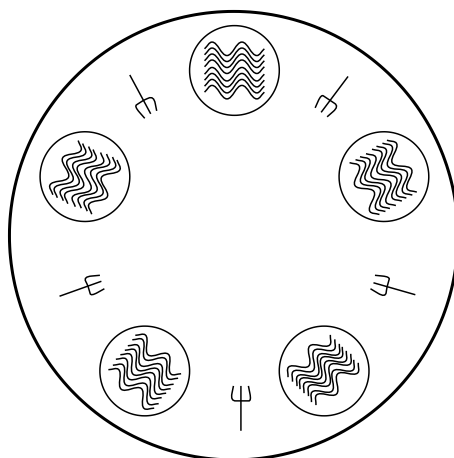
Rysunek 2.3 przedstawia wywołanie procedury `increaseByOne` na obu zsynchronizowanych wątkach, tj. wątek T_2 wykonał przydzielone mu operacje dokładnie wtedy, gdy wątek T_1 wykonał wszystkie swoje operacje. W rezultacie wartość zmiennej globalnej G wynosi 2, ponieważ procedura wywołana w wątku T_1 zwiększyła wartość zmiennej G o 1, a następnie to samo wykonała procedura wywołana w wątku T_2 , wykorzystując wynik operacji wykonanych przez wątek T_1 .

2.2.2 Problemy żywotności: *zakleszczenie*, *zagłodzenie*, *livelock*

Goetz [5] definiuje *żywotność* (ang. *liveness*) jako właściwość systemu, która zapewnia, że koniec końców pewna własność się wydarzy, czyli że system nie będzie pozostawał w stanie zawieszenia lub bezczynności. Problemy żywotności występują, gdy system nie jest w stanie kontynuować wykonywania zadań z powodu wzajemnych zależności między wątkami lub braku zasobów. Najpopularniejszymi z nich są *zakleszczenie*, *zagłodzenie* i *livelock*.

Problem uczających filozofów

Klasyczną ilustracją problemów żywotności, często przytaczaną w literaturze, jest wymyślony w 1965 roku przez Edsgera Dijkstrę *problem uczających filozofów* (ang. *dining philosophers problem*) [8]. Opisuje on pięciu filozofów siedzących przy okrągłym stole. Każdy filozof ma przed sobą talerz z posiłkiem, a między każdą sąsiadującą parą talerzy znajduje się widelec. Filozof może wykonywać jedną z dwóch czynności: albo je, albo rozmyśla. Zakłada się, że filozof może jeść tylko przy użyciu dwóch widelców. Dodatkowo, filozof może korzystać jedynie z widelców sąsiadujących z jego talerzem [1, 2].



Rysunek 2.4: Ilustracja problemu uczających filozofów

(źródło: opracowanie własne na podstawie [1])

W powyższym problemie poszczególne jego elementy stanowią abstrakcyjne reprezentacje składników systemu współbieżnego, tj. filozofowie odpowiadają procesom lub wątkom wykonującym się równolegle, które cyklicznie przechodzą pomiędzy sekcją niekrytyczną (rozmyślanie) a sekcją krytyczną (jedzenie), zaś widelce reprezentują współdzielone zasoby, do których dostęp musi być synchronizowany.

Zakleszczenie

Zakleszczeniem (ang. *deadlock*) określa się sytuację w systemie współbieżnym, w której co najmniej dwa procesy lub wątki czekają na siebie nawzajem w taki sposób, że żaden z nich nie może kontynuować wykonania [5].

Przykładem zakleszczenia w powyższym problemie jest sytuacja, w której każdy filozof bierze do jednej ręki widelec po lewej stronie swojego talerza. Aby móc zacząć jeść, filozof potrzebuje drugiego widelca, lecz każdy z nich nie posiada wolnego widelca po prawej stronie swojego talerza. Powoduje to wzajemną zależność czekania na zwolnienie zasobu, a w konsekwencji żaden z filozofów nie przystąpi do jedzenia.

Zagłodzenie

Zagłodzeniem (ang. *starvation*) wątku nazywa się sytuację, w której wątek jest stale pozbawiony dostępu do wspólnego zasobu, przez co nie może wykonać przydzielonych mu operacji [5].

Przykładem zagłodzenia w kontekście problemu uczujących filozofów jest sytuacja, w której jeden filozof nigdy nie może przystąpić do jedzenia, ponieważ sąsiadujący z nim filozofowie zawsze wygrywają w walce o wspólny zasób, jakim jest widelec (zarówno ten po lewej, jak i po prawej stronie talerza).

Livelock

Livelock jest stanem systemu współbieżnego, w którym procesy nie są zablokowane i wykonują operacje, jednak wskutek ciągłych wzajemnych reakcji nie dochodzi do postępu w realizacji ich zadań [5].

Livelock w kontekście wspomnianego wyżej problemu występuje, gdy filozofowie, próbując uniknąć zakleszczenia, zwalniają posiadane zasoby w przypadku niepowodzenia i ponawiają próby w sposób symetryczny, co prowadzi do cyklicznego wzorca zachowań. W rezultacie system pozostaje aktywny, jednak żaden z procesów nie uzyskuje jednocześnie wymaganych zasobów, przez co nie dochodzi do postępu.

2.2.3 Problemy związane z wydajnością

2.3 Modele współbieżności w środowisku JVM

Środowisko JVM oferuje wiele różnych modeli umożliwiających tworzenie współbieżnych aplikacji. Ta część poświęcona jest omówieniu najpopularniejszych podejść, w tym podstawowego (i chronologicznie pierwszego) modelu, będącego częścią Thread API, modelu opartego o wykorzystanie puli wątków, dostępnego w ramach interfejsu `ExecutorService`, a także modelu reaktywnego.

2.3.1 Klasyczny model współbieżności (Thread API)

Podstawowym modelem współbieżności w środowisku JVM są klasa `Thread` i interfejs `Runnable`, będące częścią pakietu `java.lang` i oferujące podstawowe metody umożliwiające zarządzanie wątkami. Interfejs `Runnable` reprezentuje zadanie przeznaczone do współbieżnego wykonania, natomiast klasa `Thread` odpowiada za jego uruchomienie i wykonanie [9, 10].

Klasa `Thread` jest klasą opakowującą (ang. *wrapper*) wątki systemu operacyjnego w relacji jeden do jednego. Oznacza to, że każdy obiekt tej klasy odpowiada dokładnie jednemu wątkowi systemowemu, nazywanemu *wątkiem platformowym* (ang. *platform thread*) [11]. Zarządzanie cyklem życia i przełączanie kontekstu realizowane jest przez system operacyjny, a nie przez środowisko JVM. Konsekwencją takiego podejścia jest stosunkowo wysoki koszt tworzenia wątków, zarówno pod względem czasu, jak i zużycia pamięci, co ogranicza ich skalowalność w aplikacjach wymagających obsługi dużej liczby współbieżnych zadań [12].

Jak wskazuje Lea [13], model ten opiera się na współdzielonej pamięci, co oznacza, że wątki mogą uzyskiwać dostęp do tych samych danych. W celu zapewnienia poprawności działania programu konieczne jest stosowanie mechanizmów synchronizacji. Przykładem jest obecne w środowisku JVM słowo kluczowe `synchronized` [14], które umożliwia wzajemne wykluczanie dostępu do sekcji krytycznej.

Pomimo swojej prostoty i bezpośredniego odwzorowania na mechanizmy systemu operacyjnego, model ten posiada istotne ograniczenia. Wysoki koszt tworzenia i zarządzania wątkami oraz podatność na problemy współbieżności, wspomniane w podrozdziale 2.2 Problemy występujące w systemach współbieżnych, sprawiają, że w praktyce często stosuje się bardziej zaawansowane abstrakcje, opisane niżej.

2.3.2 Model oparty o pulę wątków (ExecutorService)

Jednym z rozwiązań wspomnianego wyżej problemu było wprowadzenie w ramach JDK 1.5 interfejsu `ExecutorService` [15]. Został on wprowadzony w celu uproszczenia zarządzania współbieżnością poprzez uniezależnienie definicji zadań od ich wykonania, stanowiąc bardziej elastyczną alternatywę dla bezpośredniego użycia klasy `Thread` i interfejsu `Runnable`.

Interfejs `ExecutorService` w głównej mierze opiera się o wykorzystanie wzorca projektowego, jakim jest *Pula obiektów* (ang. *Object Pool*) [16]. W tym celu interfejs wykorzystuje pulę wątków. Takie podejście pozwala na wielokrotne wykorzystanie tych samych wątków, eliminując konieczność ich częstego tworzenia i niszczenia, co w klasycznym modelu stanowi istotne obciążenie czasowe i pamięciowe [12]. Podejście to jednak nie jest pozbawione ograniczeń. Jednym z nich jest problem odpowiedniego doboru rozmiaru puli wątków.

Określanie rozmiaru puli wątków

Rozmiar puli wątków definiuje się jako liczbę wątków zarządzanych przez mechanizm wykonawczy, które mogą być wykorzystane do równoczesnego wykonywania zadań [17]. Zdaniem Goetza [5], liczba ta powinna być wyznaczana dynamicznie, przykładowo na podstawie liczby dostępnych procesorów logicznych. W środowisku JVM wartość ta jest możliwa do wyznaczenia przy użyciu metody `availableProcessors`, dostępnej w ramach klasy `Runtime` [18].

Goetz wskazuje również, że przy ustalaniu rozmiaru puli wątków należy unikać wartości ekstremalnych. Jeśli pula wątków jest za duża, wątki mogą ze sobą konkurować o ograniczone zasoby procesora i pamięci, co może skutkować większym użyciem pamięci i możliwym wyczerpaniem zasobów, zaś zbyt mała pula wątków może ograniczać przepustowość poprzez niewykorzystanie procesorów logicznych pomimo dostępności zadań do wykonania. Ustalenie właściwego rozmiaru puli wątków zależy między innymi od dostępności zasobów i natury współbieżnie wykonywanych zadań. Goetz podaje, że optymalny rozmiar N_T puli wątków można wyrazić wzorem

$$N_T = N_{\text{CPU}} \cdot U_{\text{CPU}} \cdot \left(1 + \frac{T_W}{T_C}\right),$$

gdzie:

- N_{CPU} oznacza liczbę dostępnych procesorów logicznych,
- U_{CPU} jest wartością docelowego wykorzystania procesora ($0 \leq U_{\text{CPU}} \leq 1$),
- T_W (ang. *wait time*) reprezentuje średni czas oczekiwania wątku na możliwość kontynuowania pracy, obejmujący między innymi czas oczekiwania na ope-

racje wejścia/wyjścia, dostęp do zasobów współdzielonych lub synchronizację z innymi wątkami,

- T_C (ang. *compute time*) oznacza średni czas, w którym wątek aktywnie wykonuje operacje obliczeniowe, wykorzystując procesor.

Stosunek $\frac{T_W}{T_C}$ odzwierciedla charakter zadań wykonywanych w ramach puli wątków. Niskie wartości tego ilorazu wskazują na zadania obliczeniowe, w których dominującą rolę odgrywa czas przetwarzania (ang. *CPU-bound*), natomiast wysokie wartości są charakterystyczne dla zadań wymagających częstego oczekiwania, takich jak operacje wejścia/wyjścia (ang. *I/O-bound*).

2.3.3 Reaktywny model współbieżności (Reactive Streams, Project Reactor, RxJava)

Rozdział 3

Mechanizmy współbieżności oferowane przez Project Loom

Trzeci rozdział pracy poświęcony jest omówieniu najważniejszych mechanizmów dostępnych w ramach Project Loom – inicjatywy usprawniającej tworzenie aplikacji współbieżnych w środowisku JVM. Rozdział zawiera zwięzłą prezentację historii powstawania tego projektu, a także omówienie najważniejszych mechanizmów, które on oferuje, tj. odrębną kategorię wątków, określanych mianem wątków wirtualnych (ang. *virtual threads*), mechanizm współbieżności strukturalnej (ang. *structured concurrency*) i mechanizm przekazywania danych kontekstowych w ramach zmiennych zakresowych (ang. *scoped values*).

3.1 Wprowadzenie do Project Loom

Project Loom jest inicjatywą zapoczątkowaną pod koniec 2017 roku przez OpenJDK. W ramach projektu wyróżnia się trzy główne składowe:

- mechanizm *wirtualnych wątków* (ang. *virtual threads*), będący alternatywą dla klasycznego mechanizmu wątków platformowych i zdefiniowany w dokumencie JEP¹ 444 [19],
- mechanizm *współbieżności strukturalnej* (ang. *structured concurrency*), umożliwiający uporządkowanie pracy z wątkami poprzez dodanie *zakresów zadań* (ang. *task scopes*) i strategii zachowań w przypadku wystąpienia błędu lub przerwania działania wątku, opisany w dokumencie JEP 525² [20],

¹JEP (*JDK Enhancement Proposal*) to formalny dokument opisujący proponowane zmiany, nowe funkcjonalności lub usprawnienia platformy JDK.

²W chwili publikacji najnowsza wersja mechanizmu współbieżności strukturalnej została udostępniona w wersji *preview*, co oznacza, że jest dostępna w standardowym JDK, lecz jej składnia lub semantyka mogą ulec zmianie przed ostatecznym włączeniem do specyfikacji platformy Java.

- mechanizm przekazywania danych kontekstowych między wątkami za pomocą *zmiennych zakresowych* (ang. *scoped values*), będący alternatywą dla klasy `ThreadLocal` w systemach współbieżnych i opisany w dokumencie JEP 506 [21].

Początkowo, ideą projektu było wprowadzenie lekkich wątków użytkownika (ang. *user-mode threads*) oraz uproszczenie programowania współbieżnego, szczególnie w aplikacjach, których dużą częścią są blokujące zadania, takich jak serwery i systemy sieciowe [22]. Wraz z upływem czasu zakres projektu został rozszerzony o mechanizmy zwiększające bezpieczeństwo aplikacji współbieżnych i ustrukturyzowanie pracy z wątkami. Project Loom wciąż ewoluuje, głównie w kontekście zmian w API poszczególnych mechanizmów, lecz większość z nich jest integralną częścią JDK, dodanych w ramach oficjalnych wydań.

3.2 Wirtualne wątki (*Virtual Threads*)

Mechanizm wirtualnych wątków, we wczesnych fazach projektu określanych mianem *fibers* [22], to chronologicznie pierwsza z trzech głównych składowych projektu Loom, która pod tą nazwą po raz pierwszy pojawiła się w JDK w wersji 19 w ramach wersji poglądowej (ang. *preview*), a została oficjalnie wydana w ramach JDK 21. Mechanizm ten jest alternatywą dla wątków platformowych, opisanych w sekcji 2.3.1 Klasyczny model współbieżności (Thread API).

W celu wyjaśnienia potrzeby wprowadzenia mechanizmu wirtualnych wątków do środowiska JVM, dokument *JEP 444: Virtual Threads* [19] powołuje się na prawo Little’a [23], które wiąże ze sobą latencję, współbieżność i przepustowość w kontekście skalowalności aplikacji współbieżnych:

Dla danego czasu przetwarzania żądania (tj. latencji) liczba żądań obsługiwanych jednocześnie przez aplikację (tj. współbieżność) musi rosnać proporcjonalnie do tempa napływu żądań (tj. przepustowości).

Jak wspomniano w sekcji 2.3.1 Klasyczny model współbieżności (Thread API), wątki platformowe w środowisku JVM są zaimplementowane jako obiekty opakowujące (ang. *wrappers*) wątki systemu operacyjnego. Wątki te są zasobożerne, co uniemożliwia ich tworzenie w bardzo dużych ilościach, przez co liczba wątków często staje się czynnikiem ograniczającym skalowalność, na długo przed wyczerpaniem innych zasobów.

Wirtualne wątki umożliwiają zachowanie paradygmatu *thread-per-request*³ przy jednoczesnym osiągnięciu skalowalności porównywalnej z podejściem asynchronicznym.

Cykl życia wirtualnych wątków

Każdorazowo, gdy tworzony jest wirtualny wątek, zostaje on dodany do *kolejki wirtualnych wątków*, będącą kolejką typu FIFO (ang. *First In, First Out*). Wirtualny wątek może wykonać przydzielone mu zadanie, gdy zostanie przypisany do *wątku nośnego* (ang. *carrier thread*). Reprezentuje on wątek platformowy (tj. wątek systemu operacyjnego) po stronie platformy JVM [12]. Każdy wątek nośny ma przypisany odpowiadający mu wątek platformowy i domyślnie ich liczba jest równa liczbie dostępnych (fizycznych) procesorów logicznych [24]. W odróżnieniu od wątków platformowych i nośnych, liczba wirtualnych wątków nie jest zależna od liczby dostępnych procesorów logicznych, a w większości przypadków ją przekracza. Wynika to ze sposobu zarządzania cyklem ich życia, który został przedstawiony na rysunku 3.1.

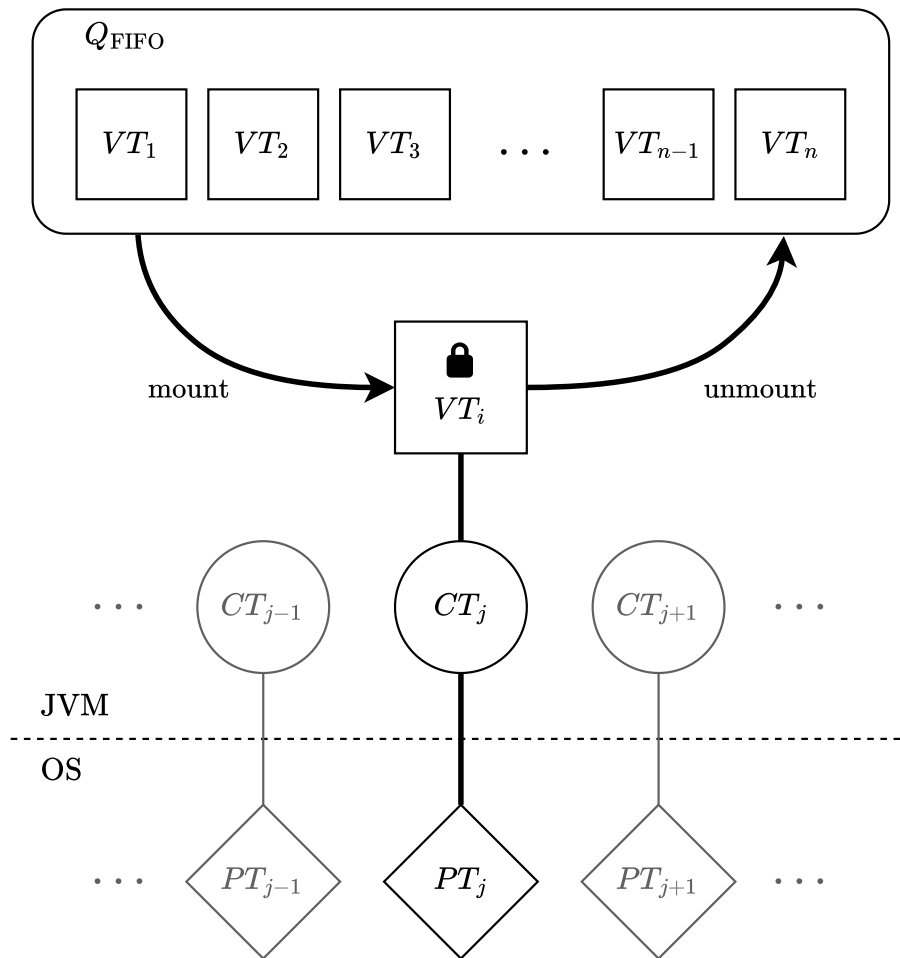
Korzystając z oznaczeń zawartych na rysunku 3.1, cykl życia wirtualnych wątków w środowisku JVM w kontekście jednego wątku nośnego można opisać następująco:

1. Każdy nowy wirtualny wątek VT zostaje dodany na koniec kolejki wirtualnych wątków Q_{FIFO}
2. Planista (ang. *scheduler*) wątków wirtualnych, obecny w środowisku JVM, bierze pierwszy wolny wątek wirtualny z kolejki przypisuje go (ang. *mount*) do wolnego wątku nośnego (CT_j)
3. Wątek nośny CT_j , odpowiadający wątkowi platformowemu PT_j , wykonuje kod przypisany do przydzielonego mu wątku wirtualnego VT_i
4. Gdy wątek wirtualny VT_i wykonuje operację blokującą (stan reprezentowany przez symbol ) , JVM zapisuje jego aktualny stan, dodaje go na koniec kolejki Q_{FIFO} , jednocześnie zwalniając (ang. *unmount*) wątek nośny CT_j
5. JVM przypisuje kolejny wątek wirtualny (tj. pierwszy w kolejce Q_{FIFO}) do wątku nośnego CT_j

Pule wirtualnych wątków

bla bla bla

³Paradygmat *thread-per-request* zakłada, że każde przychodzące żądanie jest obsługiwane przez dedykowany wątek.



Rysunek 3.1: Cykl życia wirtualnych wątków w środowisku JVM
(źródło: opracowanie własne na podstawie [12])

3.3 Współbieżność strukturalna (*Structured Concurrency*)

3.4 Zmienne zakresowe (*Scoped Values*)

Rozdział 4

Stan badań i analiza literatury

4.1 Dotychczasowe badania nad Project Loom

4.2 Charakterystyka i wyniki dotychczasowych badań

4.2.1 Obsługa dużej liczby równoczesnych żądań HTTP

4.2.2 Równoległe przetwarzanie złożonych obliczeń

4.3 Zdefiniowanie luki badawczej i kierunku własnych badań

Rozdział 5

Metodyka badań

5.1 Cel i zakres badań

5.2 Szczegółowy opis scenariuszy testowych

5.2.1 Scenariusz A: Integracja z bazą danych ze sterownikiem blokującym (JDBC) i reaktywnym (R2DBC)

5.2.2 Scenariusz B: Adaptacyjna strategia Hedged Requests z priorytetyzacją jakości danych i dynamiczną propagacją parametrów SLA

Rozdział 6

Wyniki badań i analiza

6.1 Metody pomiaru i kryteria oceny

6.2 Wyniki pomiarów dla poszczególnych scenariuszy

Uwagi do poniższych tabel:

1. Wyniki benchmarku przeprowadzonego w trybie **Throughput** (testu przepustowości), tj. w tabelach 6.1, 6.2 i 6.3, podane są jako liczba operacji na sekundę (wykresy przedstawione na rysunkach 6.1, 6.2 i 6.3 prezentują te dane na osi pionowej w jednostce oznaczonej jako ops/s). Operacje wykonane w ramach każdego benchmarku opisane zostały w rozdziale X.
2. Wyniki w tabelach wspomnianych w punkcie 1. przedstawione są w formacie $S \pm E$, gdzie S oznacza wynik, a E – błąd pomiaru.
3. Wyniki we wszystkich tabelach zapisane zostały z kropką jako separatorem dziesiętnym, z dokładnością do części tysięcznych, tj. 0.001.

6.3 Analiza porównawcza i interpretacja wyników

Poz. współb.	JDBC + PT	R2DBC	JDBC + VT
16	415.881 ± 8.497	729.356 ± 5.935	733.642 ± 13.414
32	230.098 ± 7.511	451.219 ± 2.474	489.411 ± 2.623
64	116.672 ± 4.463	240.370 ± 4.193	250.862 ± 0.876
256	28.871 ± 1.067	67.178 ± 0.352	66.336 ± 0.378
1024	6.694 ± 0.306	17.139 ± 0.105	16.782 ± 0.111

Tabela 6.1: Wartości przepustowości dla zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

Poz. współb.	JDBC + PT	R2DBC	JDBC + VT
16	142.817 ± 24.898	174.474 ± 36.850	159.069 ± 33.964
32	107.766 ± 14.097	122.683 ± 0.635	157.456 ± 29.941
64	55.572 ± 5.507	74.120 ± 10.150	70.235 ± 4.447
256	14.765 ± 1.268	19.392 ± 2.232	19.884 ± 1.825
1024	3.736 ± 0.365	4.600 ± 0.031	5.042 ± 0.379

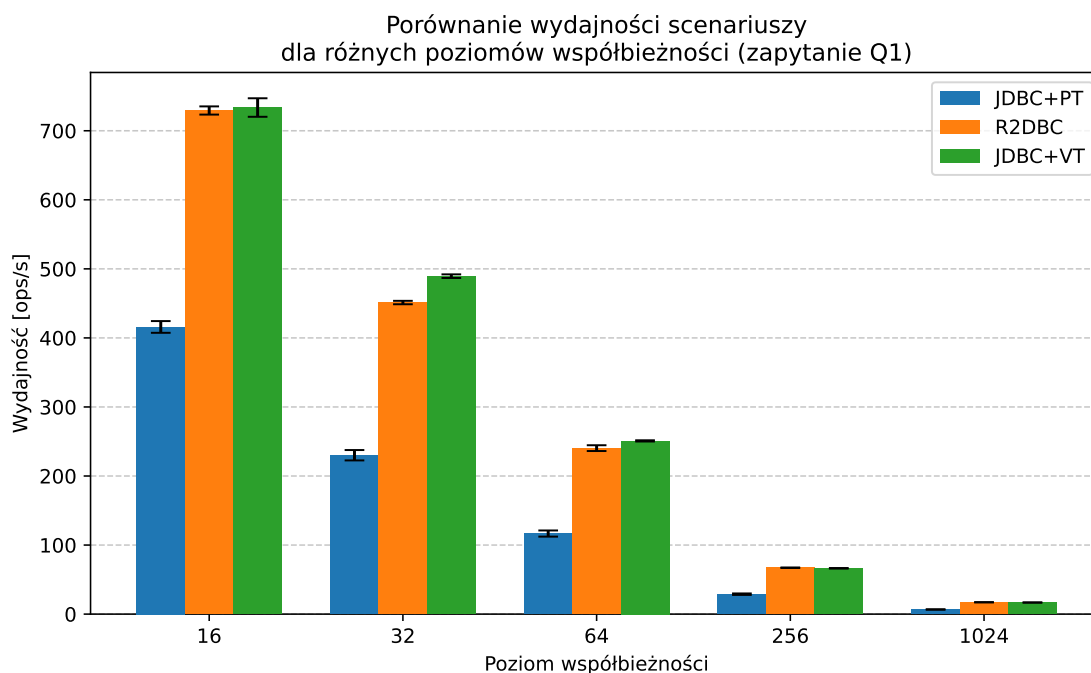
Tabela 6.2: Wartości przepustowości dla zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

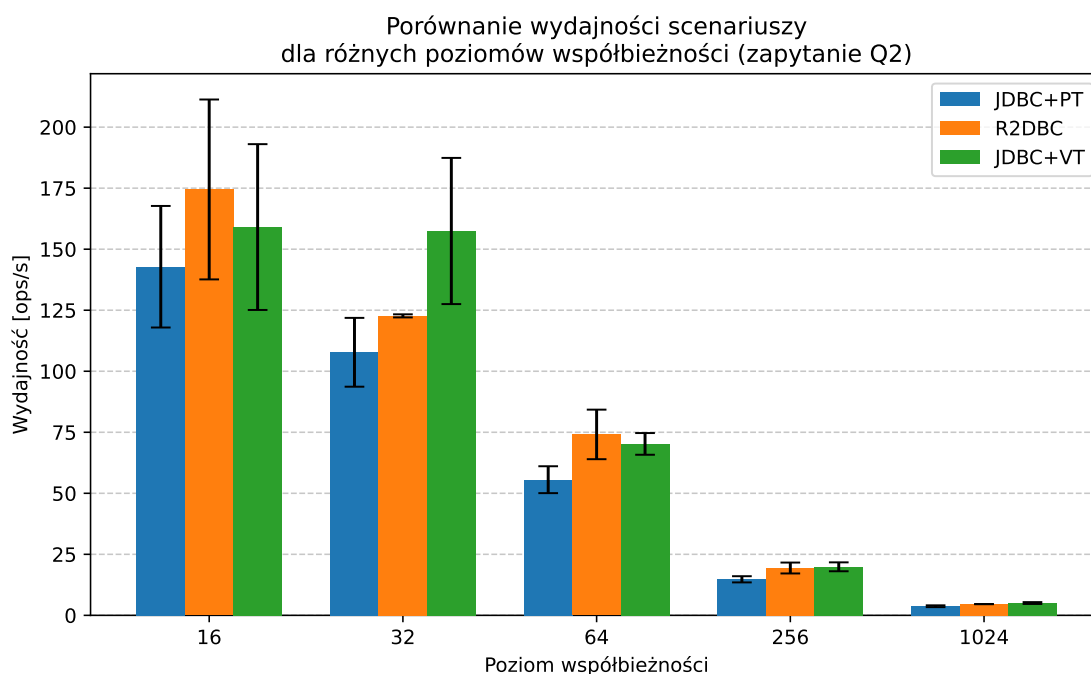
Poz. współb.	JDBC + PT	R2DBC	JDBC + VT
16	4.943 ± 0.020	5.003 ± 0.027	4.940 ± 0.027
32	2.667 ± 0.009	2.705 ± 0.011	2.673 ± 0.010
64	1.319 ± 0.004	1.332 ± 0.006	1.314 ± 0.008
256	0.330 ± 0.002	0.335 ± 0.001	0.330 ± 0.002
1024	0.187 ± 0.001	0.084 ± 0.001	0.192 ± 0.001

Tabela 6.3: Wartości przepustowości dla zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności

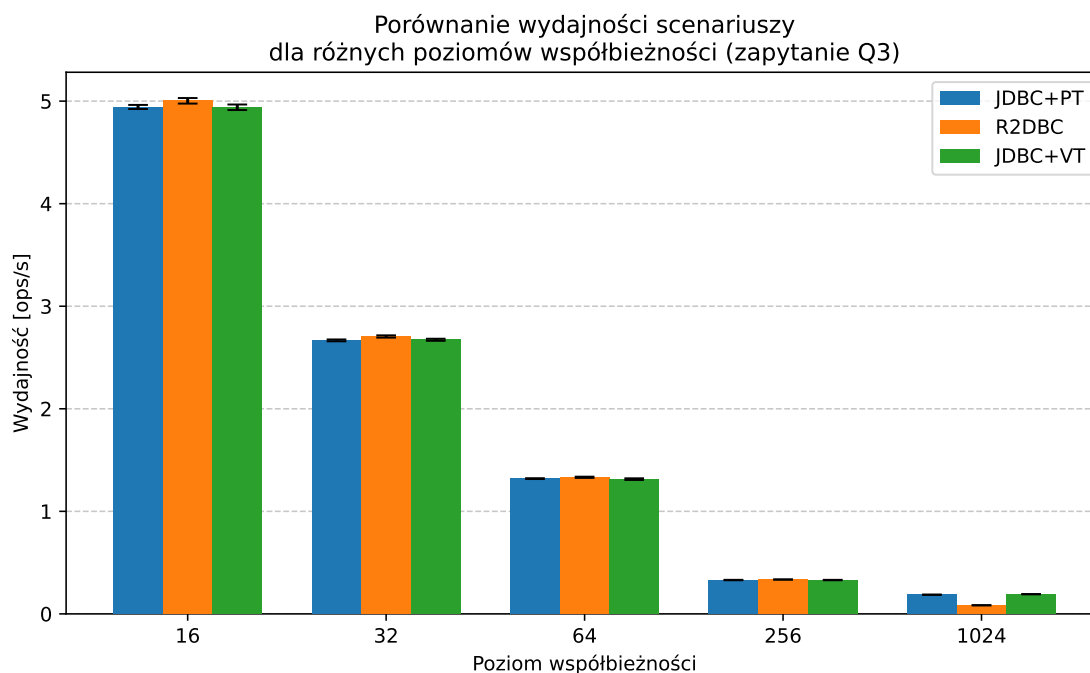
(źródło: opracowanie własne)



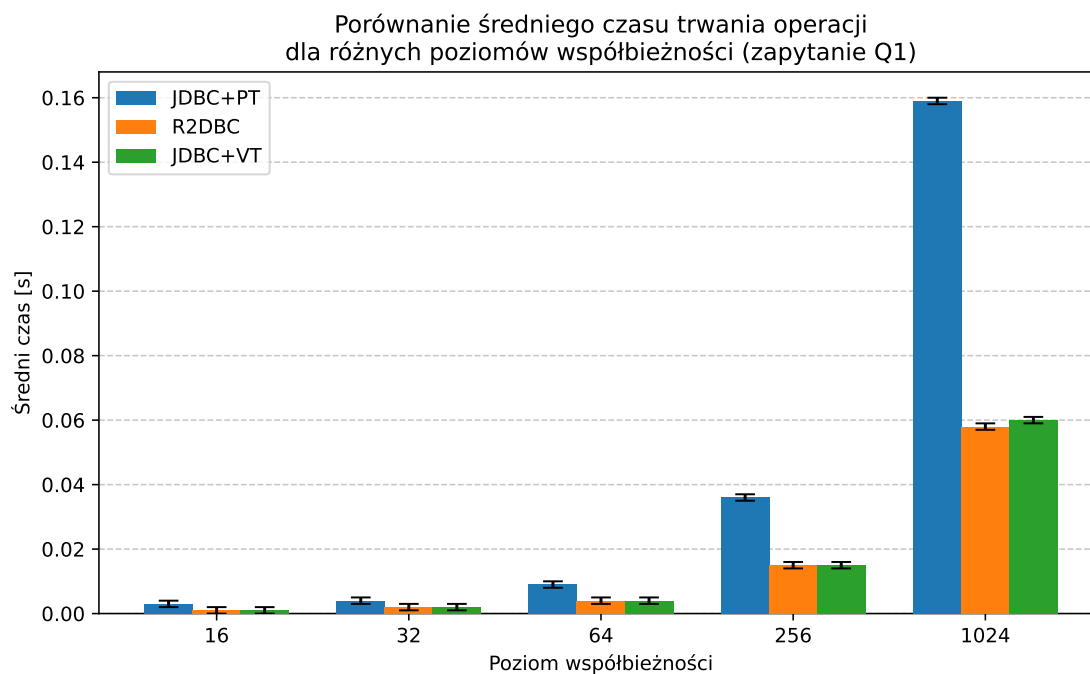
Rysunek 6.1: Wykres kolumnowy przepustowości dla zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności
(źródło: opracowanie własne)



Rysunek 6.2: Wykres kolumnowy przepustowości dla zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności
(źródło: opracowanie własne)



Rysunek 6.3: Wykres kolumnowy przepustowości dla zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności



Rysunek 6.4: Wykres kolumnowy średniego czasu zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

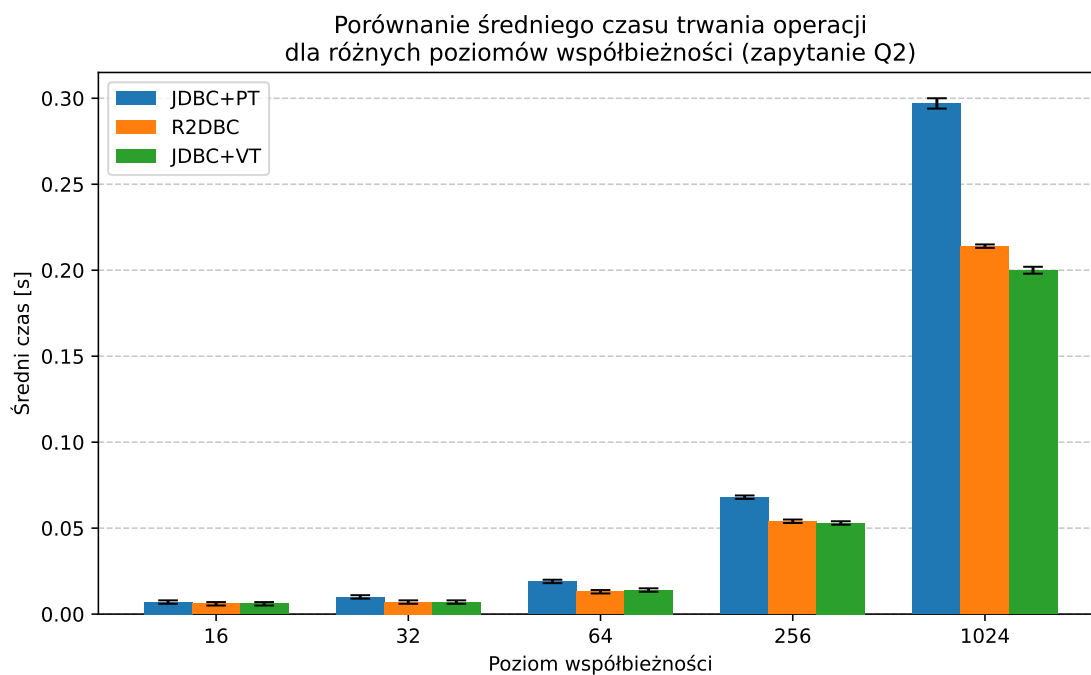
Poz. współb.	Metryka	JDBC + PT	R2DBC	JDBC + VT
16	AVG	0.003 ± 0.001	0.001 ± 0.001	0.001 ± 0.001
	P50	0.003	0.001	0.001
	P95	0.003	0.002	0.002
	P99	0.004	0.002	0.002
32	AVG	0.004 ± 0.001	0.002 ± 0.001	0.002 ± 0.001
	P50	0.004	0.002	0.002
	P95	0.005	0.003	0.002
	P99	0.006	0.004	0.003
64	AVG	0.009 ± 0.001	0.004 ± 0.001	0.004 ± 0.001
	P50	0.009	0.004	0.004
	P95	0.010	0.005	0.005
	P99	0.011	0.006	0.006
256	AVG	0.036 ± 0.001	0.015 ± 0.001	0.015 ± 0.001
	P50	0.035	0.015	0.015
	P95	0.039	0.017	0.018
	P99	0.043	0.020	0.020
1024	AVG	0.159 ± 0.001	0.058 ± 0.001	0.060 ± 0.001
	P50	0.158	0.057	0.058
	P95	0.177	0.065	0.066
	P99	0.187	0.070	0.071

Tabela 6.4: Wartości średniego czasu i latencji P50, P95 i P99 dla zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności
(źródło: opracowanie własne)

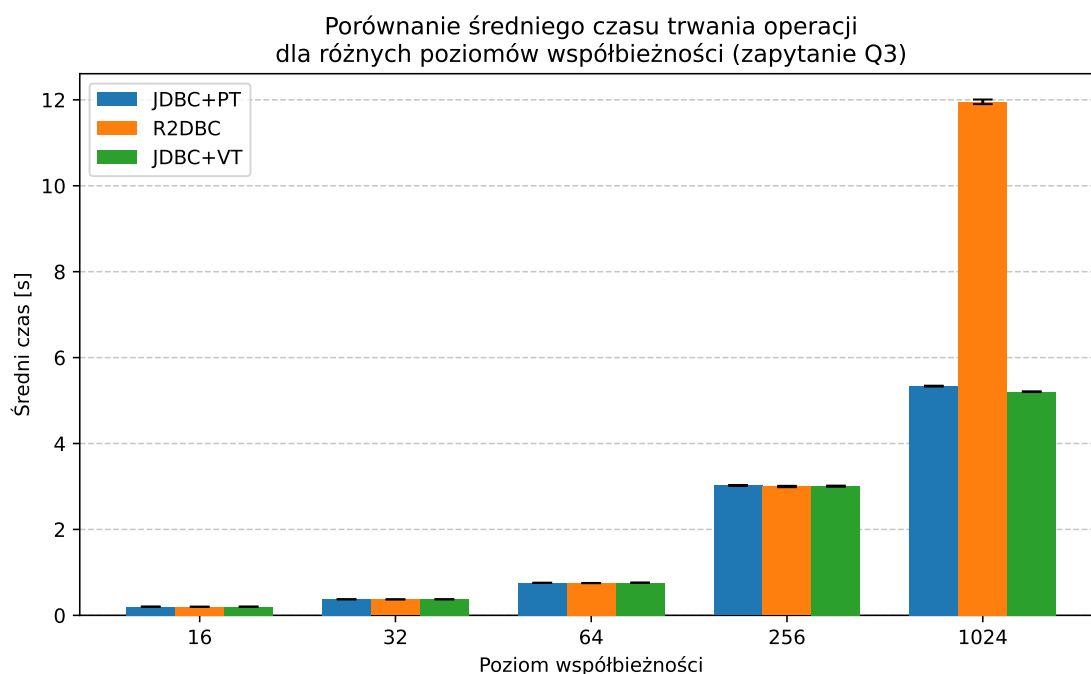
Poz. współb.	Metryka	JDBC + PT	R2DBC	JDBC + VT
16	AVG	0.007 ± 0.001	0.006 ± 0.001	0.006 ± 0.001
	P50	0.008	0.007	0.007
	P95	0.010	0.008	0.008
	P99	0.012	0.010	0.011
32	AVG	0.010 ± 0.001	0.007 ± 0.001	0.007 ± 0.001
	P50	0.010	0.008	0.008
	P95	0.011	0.009	0.009
	P99	0.014	0.011	0.011
64	AVG	0.019 ± 0.001	0.013 ± 0.001	0.014 ± 0.001
	P50	0.019	0.014	0.014
	P95	0.021	0.016	0.016
	P99	0.025	0.020	0.022
256	AVG	0.068 ± 0.001	0.054 ± 0.001	0.053 ± 0.001
	P50	0.071	0.054	0.052
	P95	0.077	0.059	0.056
	P99	0.096	0.081	0.079
1024	AVG	0.297 ± 0.003	0.214 ± 0.001	0.200 ± 0.002
	P50	0.299	0.212	0.202
	P95	0.329	0.236	0.223
	P99	0.364	0.264	0.257

Tabela 6.5: Wartości średniego czasu i latencji P50, P95 i P99 dla zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)



Rysunek 6.5: Wykres kolumnowy średniego czasu zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności
(źródło: opracowanie własne)



Rysunek 6.6: Wykres kolumnowy średniego czasu zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności
(źródło: opracowanie własne)

Poz. współb.	Metryka	JDBC + PT	R2DBC	JDBC + VT
16	AVG	0.202 ± 0.001	0.200 ± 0.001	0.202 ± 0.001
	P50	0.200	0.199	0.200
	P95	0.217	0.216	0.218
	P99	0.227	0.226	0.228
32	AVG	0.374 ± 0.001	0.372 ± 0.001	0.374 ± 0.001
	P50	0.371	0.369	0.372
	P95	0.400	0.394	0.394
	P99	0.412	0.412	0.406
64	AVG	0.757 ± 0.003	0.751 ± 0.003	0.760 ± 0.003
	P50	0.753	0.748	0.755
	P95	0.784	0.778	0.793
	P99	0.820	0.820	0.830
256	AVG	3.023 ± 0.012	2.999 ± 0.016	3.007 ± 0.013
	P50	3.012	2.991	3.001
	P95	3.091	3.078	3.095
	P99	3.176	3.311	3.148
1024	AVG	5.336 ± 0.010	11.954 ± 0.054	5.206 ± 0.009
	P50	5.335	11.929	5.201
	P95	5.377	12.148	5.234
	P99	5.385	12.231	5.260

Tabela 6.6: Wartości średniego czasu i latencji P50, P95 i P99 dla zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności

(źródło: opracowanie własne)

Rozdział 7

Wnioski i rekomendacje

- 7.1 Odpowiedzi na pytania badawcze
- 7.2 Rekomendacje i wzorce zastosowań mechanizmów Project Loom
- 7.3 Ograniczenia badań i propozycje dalszych badań

Rozdział 8

Podsumowanie

Stworzenie aplikacji *Grocery Scanner* wymagało ogromnego nakładu czasowego. Proces jej powstawania był pełen różnych koncepcji i pomysłów na rozbudowę aplikacji. Wiele z nich często podlegało zmianom, lecz efekt końcowy pozwala stwierdzić, że aplikacja spełnia wyrażone we wstępie założenia. Możliwe jest jednak poszerzanie jej o dodatkowe funkcjonalności. Przykładowymi planami na rozwój aplikacji są:

- *możliwość logowania i rejestracji za pomocą konta w takich serwisach, jak Google i Facebook*, co usprawni zaimplementowany już proces, a w konsekwencji pozwoli na powiększenie liczby użytkowników korzystających z aplikacji,
- *system zgłaszania profili użytkowników*, który pozwoli na ograniczenie szkodliwej działalności w aplikacji poprzez ograniczanie lub blokowanie dostępu,
- *rozszerzenie listy preferencji żywieniowych*, która ułatwi korzystanie z aplikacji większej liczbie użytkowników,
- *pobieranie informacji o produktach z innych źródeł*, co pozwoli na wzrost jakości danych, a także w dużym stopniu ograniczy przypadki, w których aplikacja zwraca informację o braku danych skanowanego produktu.

Warto zaznaczyć, że funkcjonalności zawarte w powyższej liście nie zostały zaimplementowane z uwagi na fakt, że wykraczały poza podstawowe założenia co do funkcjonalności aplikacji opisanych we wstępie pracy.

Proces tworzenia niniejszej pracy przede wszystkim pozwolił na pogłębienie wiedzy. W tej kwestii szczególną uwagę należy zwrócić uwagę na aspekt „techniczny”. Tworzenie aplikacji *Grocery Scanner* pozwoliło na poznanie nieużywanych wcześniej przez autora pracy bibliotek i narzędzi, jak również umożliwiło poszerzenie wiedzy na temat poznanych wcześniej rozwiązań. Sam proces tworzenia aplikacji na urządzenia mobilne okazał się być niezwykle pouczający, zarówno w kontekście poszerzania wiedzy technicznej, jak i rozwoju kariery zawodowej.

Bibliografia

- [1] Andrew S. Tanenbaum i Herbert Bos. *Modern Operating Systems*. Wydanie V. Pearson Education, 2023, s. 85–178.
- [2] Mordechai Ben-Ari. *Principles of Concurrent Programming*. Prentice-Hall International, 1982, s. 1–15, 18–27, 29–43, 51–55, 109–116.
- [3] Maurice Herlihy i Nir Shavit. *The Art of Multiprocessor Programming*. Elsevier, 2012, s. 6–10, 21–44.
- [4] Vincent Gramoli. *More than you ever wanted to know about synchronization: Synchrobench, measuring the impact of the synchronization on concurrent algorithms*. URL: <http://sydney.edu.au/engineering/it/~gramoli/doc/pubs/gramoli-synchrobench.pdf> (dostęp 27.03.2026).
- [5] Brian Goetz i in. *Java Concurrency in Practice*. Pearson Education, 2006.
- [6] Abraham Silberschatz, Peter B. Galvin i Greg Gagne. *Podstawy systemów operacyjnych, Tom I*. Wydanie X (I w WN PWN). Wydawnictwo Naukowe PWN, 2021.
- [7] Microsoft Learn. *Warunki wyścigu i zakleszczenia*. Przetłumaczone oryginalnie z języka angielskiego. URL: <https://learn.microsoft.com/pl-pl/troubleshoot/developer/visualstudio/visual-basic/language-compilers/race-conditions-deadlocks> (dostęp 28.03.2026).
- [8] Edsger W. Dijkstra. *Twenty-eight years*. Transkrypcja odręcznych notatek z 1965 roku. URL: <https://www.cs.utexas.edu/~EWD/transcriptions/EWD10xx/EWD1000.html> (dostęp 28.03.2026).
- [9] Dokumentacja klasy `java.lang.Thread` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Thread.html> (dostęp 5.04.2026).
- [10] Dokumentacja interfejsu `java.lang.Runnable` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Runnable.html> (dostęp 5.04.2026).
- [11] Jeff Friesen. *Java Threads and the Concurrency Utilities*. Apress, 2015, s. 3–49.

- [12] Ron Veen i David Vlijmincx. *Virtual Threads, Structured Concurrency, and Scoped Values: Explore Java's New Threading Model*. Apress Berkeley, CA, 2024.
- [13] Doug Lea. *Concurrent Programming in Java: Design Principles and Patterns, Second Edition*. Addison Wesley, 1999, s. 5–31, 32–37.
- [14] Dokumentacja słowa kluczowego `synchronized` w specyfikacji języka Java. URL: <https://docs.oracle.com/javase/specs/jls/se25/html/jls-14.html#jls-14.19> (dostęp 5.04.2026).
- [15] Dokumentacja interfejsu `java.util.concurrent.ExecutorService` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/ExecutorService.html> (dostęp 5.04.2026).
- [16] Scott L. Bain. *The Design Patterns Companion*. Project Management Institute, 2020, s. 48–49.
- [17] Dokumentacja klasy `java.util.concurrent.ThreadPoolExecutor` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/util/concurrent/ThreadPoolExecutor.html> (dostęp 6.04.2026).
- [18] Dokumentacja klasy `java.lang.Runtime` dla JDK w wersji 25. URL: <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/Runtime.html> (dostęp 6.04.2026).
- [19] OpenJDK. *JEP 444: Virtual Threads*. URL: <https://openjdk.org/jeps/444> (dostęp 1.11.2025).
- [20] OpenJDK. *JEP 525: Structured Concurrency (Sixth Preview)*. URL: <https://openjdk.org/jeps/525> (dostęp 1.11.2025).
- [21] OpenJDK. *JEP 506: Scoped Values*. URL: <https://openjdk.org/jeps/506> (dostęp 1.11.2025).
- [22] Ron Pressler (OpenJDK). *Project Loom: Fibers and Continuations for the Java Virtual Machine*. URL: <https://cr.openjdk.org/~rpressler/loom/Loom-Proposal.html> (dostęp 12.12.2025).
- [23] John D.C. Little i Stephen C. Graves. *Little's Law*. URL: <https://web.archive.org/web/20100215131801/http://web.mit.edu/sgraves/www/papers/Little's%20Law-Published.pdf> (dostęp 26.01.2015).
- [24] Alan Bateman (OpenJDK). *Project Loom: The challenges of introducing Virtual Threads to the Java Platform*. URL: <https://cr.openjdk.org/~alanb/jvmls2023/loom-jvmls2023.pdf> (dostęp 11.03.2026).

Spis rysunków

2.1	Rozróżnienie procesów i wątków w systemie operacyjnym	10
2.2	Ilustracja zjawiska <i>race condition</i> na podstawie wywołania procedury <code>increaseByOne</code>	12
2.3	Wywołanie procedury <code>increaseByOne</code> z wykorzystaniem synchronizacji wątków	12
2.4	Ilustracja problemu uczających filozofów	13
3.1	Cykl życia wirtualnych wątków w środowisku JVM	22
6.1	Wykres kolumnowy przepustowości dla zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności	29
6.2	Wykres kolumnowy przepustowości dla zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności	29
6.3	Wykres kolumnowy przepustowości dla zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności	30
6.4	Wykres kolumnowy średniego czasu zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności	30
6.5	Wykres kolumnowy średniego czasu zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności	33
6.6	Wykres kolumnowy średniego czasu zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności	33

Spis tabel

6.1	Wartości przepustowości dla zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności	28
6.2	Wartości przepustowości dla zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności	28
6.3	Wartości przepustowości dla zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności	28
6.4	Wartości średniego czasu i latencji P50, P95 i P99 dla zapytania Q1 w zależności od poziomu współbieżności i wariantu modelu współbieżności	31
6.5	Wartości średniego czasu i latencji P50, P95 i P99 dla zapytania Q2 w zależności od poziomu współbieżności i wariantu modelu współbieżności	32
6.6	Wartości średniego czasu i latencji P50, P95 i P99 dla zapytania Q3 w zależności od poziomu współbieżności i wariantu modelu współbieżności	34

Spis kodów źródłowych

1	Uruchamianie wirtualnego wątku	7
---	--	---